

LLM-Driven Testing: Strategies & Patterns

Testing by Analogy

Prompt Patterns

Risk Management

TESTING PER ANALOGIA

Analogia

Definizione

L'LLM non esegue il codice (non è un interprete) e non ragiona deduttivamente (non è un logico puro).

L'LLM lavora per **ANALOGIA PROBABILISTICA**.

Quando chiedi un test per login(user, pass), l'LLM non analizza la funzione. Recupera dalla sua memoria milioni di esempi di test di login e genera qualcosa di statisticamente simile.

Confronto Paradigmi

Black Box

Basato su specifiche.

"Se clicco qui, succede questo."

White Box

Basato sulla struttura del codice.

"Coprire il ramo IF alla riga 10."

Analogia

Basato su esperienza.

"Questo codice assomiglia a un sistema di pagamento, quindi devo testare valute negative."

Potenza e Rischi dell'Analogia

Superpoteri

L'LLM può suggerire test per scenari che non hai menzionato esplicitamente, solo perché "di solito" quel tipo di codice li richiede (es. SQL Injection su campi di testo).

Kryptonite

Se il tuo codice usa un pattern unico o non standard, l'LLM "allucinerà" un test standard che non funziona nel tuo contesto specifico, forzando l'analogia dove non esiste.

How-To

PROMPT PATTERNS

02

Pattern 1: The Persona Strategy

Assegnare un ruolo specifico riduce lo spazio di ricerca dell'LLM e setta il tono delle asserzioni.

// BAD PROMPT “Scrivi un test per questa funzione.” *// RISULTATO* Test generico, framework misti, poche asserzioni

“Agisci come un *// PERSONA
PROMPT* Specializzato in sicurezza. Usa ... e ... Il tuo obiettivo è trovare vulnerabilità nel codice”

Pattern 2: Context Loading

Per sfruttare l'analogia, l'LLM deve "vedere" le dipendenze.

Prompt Template

```
# CONTEXT
Ecco la classe da testare: `UserService`

[CODE]

Ecco l'interfaccia usata: `UserRepository`

[CODE]

Ecco il DTO: `UserDto`
```

Senza le interfacce, l'LLM inventerà metodi (Allucinazioni) per far compilare il test.

Pattern 3: Few-Shot (Style Transfer)

Non descrivere lo stile di coding. Mostralo.

Prompt Template

Usa questo stile di test come riferimento:

[ESEMPIO INPUT]:

```
class Calc { add(a, b) }
```

[ESEMPIO OUTPUT]:

```
@Test
```

```
@DisplayName("Should return the sum")
```

L'LLM copierà le annotazioni (@DisplayName) e la libreria di asserzioni (AssertJ) per imitazione.

Pattern 4: Chain of Thought (CoT)

Forza l'LLM a **pianificare** prima di codificare. Migliora drasticamente la logica.

CoT Prompt Strategy

"Prima di generare il codice Java: 1. Analizza il metodo `processPayment`. 2. Elenca tutti i possibili scenari di errore (network down, insufficient funds). 3. Definisci i valori di input per i casi limite. 4. Solo dopo aver fatto questo elenco, scrivi i test."

Pattern 5: Iterative Refinement

Non accettare mai la prima risposta. Usa un processo a più step.

Step 1: Draft

"Genera i test per la classe X."

Step 2: Critique

"Critica questi test. Manca qualche edge case? Sono leggibili?"

Step 3: Refine

"Riscrivi i test incorporando i miglioramenti identificati."

ESEMPI PRATICI

Esempio 1: Boundary Value

// LOGICA DI BUSINESS

```
public double getTicketPrice(int age) {  
    // Specs: "Under 12 years old is free"  
    if (age < 12) {  
        return 0.0;  
    }  
    return 15.0;  
}
```

PROMPT: "Analizza i confini per `age`. Genera test per il valore critico 12."

Esempio 2: String Formatting

// LOGICA DI BUSINESS

```
public String getRegion(String code) {  
    // Codes starting with "01" are US  
    if (code.startsWith("01")) {  
        return "US";  
    }  
    return "GLOBAL";  
}
```

PROMPT: "Testa la logica dei prefissi, focalizzandoti sul codice regione '01'."

Esempio 3: System Thresholds

// LOGICA DI BUSINESS

```
public void checkLoad(int percentage) {  
    // Trigger alarm over 93%  
    if (percentage > 93) {  
        alertService.trigger();  
    }  
}
```

PROMPT: "Genera test per la soglia di carico critica a 93%."

// GENERATED TESTS

```
@Test  
void testSafeLoad() {  
    checkLoad(93); // Safe  
    verifyNoInteractions(alertService);  
}
```

```
@Test  
void testCriticalLoad() {  
    checkLoad(94); // Critical  
    verify(alertService).trigger();  
}
```

Critical Analysis

LIMITAZIONI & RISCHI

Limitazione 1: Allucinazioni

Il problema

L'LLM inventa metodi di libreria che "suonano bene" ma non esistono.

// CODICE ALLUCINATO

```
Mockito.verifyOnce(mock);           // Non esiste!
```

```
Assert.assertEquals(d1, d2);        // Non esiste!
```

```
Mockito.verify(mock, times(1));      // esiste!
```

Causa: L'analogia fonde sintassi di librerie diverse (es. Jest vs JUnit).

Limitazione 2: People Pleasing

L'AI vuole compiacerti e tende a dare ragione al tuo codice, anche se sbagliato.

// TUO CODICE BUGGATO

```
int add(a, b) {  
    return a - b;  
}
```

// AI TEST

```
@Test  
void testAdd() {  
    // L'AI adatta il test al bug!  
    assertEquals(0, add(2, 2));  
}
```

Limitazione 3: Illusione della Copertura

100%

Line Coverage

Un LLM può generare test che eseguono tutto il codice ma non verificano nulla (Test Tautologici).

`assertEquals(result, result)` → Utile per la coverage metrics, inutile per la qualità.

Limitazione 4: Catastrophic Forgetting

In conversazioni lunghe, l'LLM "dimentica" le istruzioni iniziali (es. "Usa JUnit 5").

Sintomo

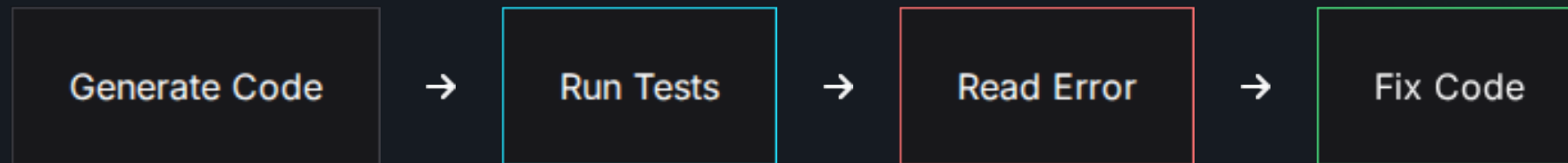
Inizia a usare JUnit 4 o mescola librerie di mocking diverse.

Soluzione

Ripetere le constraints (System Prompt) ad ogni interazione critica.

Agenti Autonomi (TestGen / Cover Agent)

Come superare i limiti? Usando Agenti con feedback loop.



Strumenti come **CodiumAI** e **Meta TestGen** automatizzano questo ciclo.

Recap: I 3 Pilastri per il Successo

1. Context is King

Fornisci interfacce e dipendenze.
Senza contesto, l'analogia
fallisce.

2. Prompt Engineering

Usa Persona, CoT e Few-Shot.
Non chiedere solo "scrivi un test".

3. Trust but Verify

Human in the loop. Code review
obbligatoria per evitare sicofania.